

Лабораторная работа № 13

Приближенное вычисление преобразования Фурье

При решении широкого круга прикладных задач, в частности в цифровом спектральном анализе, в цифровом моделировании фильтров, распознавании образов, анализе речевых сигналов, а также при решении многих задач численного анализа, возникает необходимость вычисления интегралов вида

$$\int_a^b f(x) e^{-i\omega x} dx, \quad (1)$$

где ω — произвольное действительное число. Для вычисления этого интеграла можно применить многие известные классические правила интегрирования, такие, например, как формула трапеций, парабол и др., основанные на формулах Котеса, Гаусса и т. д. Однако все эти формулы имеют существенный недостаток.

Формулы, которые упоминались выше, получают с помощью замены интегрируемой функции алгебраическим многочленом невысокого порядка на всем отрезке интегрирования или его частях. Поэтому следует ожидать, что они будут давать хорошую точность, если интегрируемая функция достаточно гладка и не очень быстро меняется. В интеграле (1) интегрируемыми функциями являются произведения $f(x) \sin \omega x$ и $f(x) \cos \omega x$. Если параметр ω — большое число, то функции $\cos \omega x$ и $\sin \omega x$ быстро колеблются, и для того чтобы достаточно точно проследить за изменением произведений $f(x) \sin \omega x$ и $f(x) \cos \omega x$ даже при медленно меняющейся функции $f(x)$, нужно взять в квадратурной формуле большое число узлов. В результате вычисления могут стать трудными или даже невыполнимыми.

Чтобы построить квадратурную формулу, пригодную для вычисления интеграла (1) в широком диапазоне изменения параметра ω , необходимо учесть множители $\sin \omega x$ и $\cos \omega x$ ($e^{-i\omega x}$) в подынтегральной функции.

В прикладных задачах чаще всего требуется вычислить интеграл (1) не при одном, а сразу при нескольких значениях параметра ω вида $\omega_k = k\Delta\omega$, $k = 0, \dots, n-1$. Все это при-

водит к необходимости получить такую квадратурную формулу и алгоритм ее реализации, в которых вычисления проводятся не последовательно для каждого значения параметра ω_k а сразу для всей совокупности значений этого параметра.

В данной лабораторной работе используется квадратурная формула вида

$$I(\omega) = \sum_{j=0}^{n-1} f_j A_j, \quad (2)$$

где

$$A_j = \int_{x_j}^{x_{j+1}} \exp(-i\omega x) dx = \frac{1 - e^{-i\omega h}}{i\omega} e^{-i\omega x_j};$$

$x_j = a + jh$, $h = \frac{b-a}{n}$, $j = 0, 1, \dots, n-1$, x_j – узлы равномерной сетки на $[a, b]$; $f_j = f(x_j)$ значения функции $f(x)$ в узлах сетки.

Для значений $\omega_k = \frac{2\pi k}{b-a}$, $\Delta\omega = \frac{2\pi}{b-a}$ формулу (2) можно переписать в виде

$$I(\omega) = \frac{1 - e^{-i\omega_k h}}{i\omega_k h} h e^{-i\omega_k a} F(k), \quad (3)$$

где

$$F(k) = \sum_{j=0}^{n-1} f_j e^{-i \frac{2\pi k j}{n}}. \quad (4)$$

Переход от величин f_j к величинам $F(k)$ ($j; k = 0, 1, \dots, n-1$) называется *дискретным преобразованием Фурье* (ДПФ).

Дискретное преобразование Фурье является составной частью решения многих прикладных задач. Замечательным свойством дискретного преобразования Фурье является возможность его обращения, т. е. восстановления величин f_j по известным значениям $F(k)$ ($j, k, 1, \dots, n-1$). Эта операция осуществляется по формулам обратного преобразования Фурье

$$f_j = \frac{1}{n} \sum_{k=0}^{n-1} F(k) e^{i \frac{2\pi k j}{n}}. \quad (5)$$

Непосредственно осуществление дискретных преобразований Фурье по формулам (4) или (5) требует $O(n^2)$ арифметических операций. Для сокращения объема вычислений были разработаны алгоритмы быстрого преобразования Фурье (БПФ). Основная идея *алгоритма быстрого преобразования Фурье* основана на том, что при составном n в правой части (4) или (5) можно выделить такие группы слагаемых, которые дают вклад во многие коэффициенты $F(k)$. Наибольшее распространение получил алгоритм БПФ, разработанный для случая $n = 2^m$. Для его реализации требуется $O(n \log_2 n)$ арифметических операций.

Процедура FF производит быстрое преобразование Фурье массива из n комплексных чисел.

procedure FF (var xr, xi, yr, yi: TMas; const n, ind: Integer);

В х о д н ы е п а р а м е т р ы : xr, xi – массивы размерности n , содержат соответственно действительные и мнимые части элементов заданного массива; yr, yi – рабочие массивы размерности n ; n – количество заданных комплексных чисел (целая степень двойки);

ind – признак, указывающий направление преобразования Фурье:

$ind > 0$ – производится прямое преобразование;

$ind < 0$ – производится обратное преобразование.

В ы х о д н ы е п а р а м е т р ы : действительная и мнимая части дискретного преобразования Фурье хранятся в массивах xr и xi , соответственно

Схема алгоритма приведена на рис 43.

Текст процедуры:

procedure FF(var xr, xi, yr, yi: TMas; const n, ind: Integer);

var

i, ni, j, jm, k, k1, m, mm, m2, log2, l, l1, it: Integer;

xa, xb, w, si, co: Real;

begin

k := n;

log2 := 0;

repeat

k := k div 2;

log2 := log2 + 1;

until (k < 2);

```

mm := 1;
{ cycl 1 }
for m := 1 to log2 do
begin
  m2 := mm*2;
  k1 := trunc(power(2,(log2 - m)) - 1);
  l1 := mm - 1;
  { cycl 2 }
  for k := 1 to k1 + 1 do
  begin
    { cycl 3 }
    for l := 1 to l1 + 1 do
    begin
      j := m2*(k - 1) + l;
      i := mm*(k - 1) + l;
      w := pi*(l - 1)/mm;
      si := ind*sin(w);
      co := cos(w);
      ni := trunc(power(2,(log2 - 1)) + i);
      jm := trunc(j + power(2, (m - 1)));
      xa := xr[ni]*co + xi[ni]*si;
      xb := xi[ni]*co - xr[ni]*si;
      yr[j] := xr[i] + xa;
      yi[j] := xi[i] + xb;
      yr[jm] := xr[i] - xa;
      yi[jm] := xi[i] - xb;
    end;
    { cycl 3 }
  end;
  { cycl 2 }
{ cycl 5 }
for it := 1 to n do
begin
  xr[it] := yr[it];
  xi[it] := yi[it];
end;
{ cycl 5 }
mm := m2;
end;
{ cycl 1 }
if ind < 0 then exit;
{ cycl 4 }
for i := 1 to n do
begin
  xr[i] := xr[i]/n;
  xi[i] := xi[i]/n;
end;
{ cycl 4 } end;

```

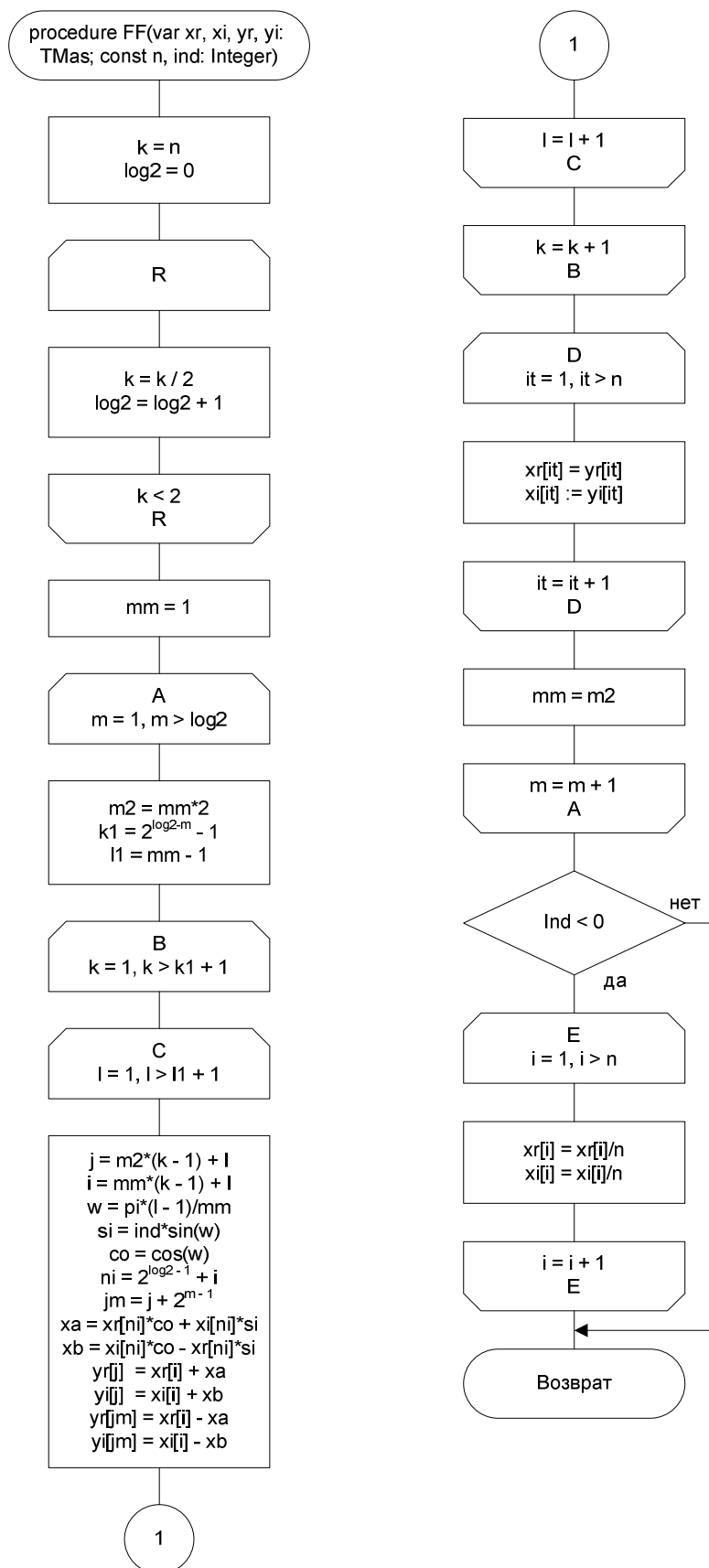


Рисунок 43 – Схема алгоритма прямого и обратного быстрого преобразования Фурье.

Процедура IFF реализует алгоритм вычисления интеграла (1) по формулам (2) – (4)

procedure IFF (const a, b: Real; var ar, ai, xr, xi: TMas; const n: Integer;

eps: Real; const ip: Integer);

В х о д н ы е п а р а м е т р ы : a, b – концы интервала интегрирования; ar, ai – массивы размерности n , содержат соответственно действительную и мнимую части значений функции $f(x)$ в равноотстоящих узлах $x_i = a + i(b - a)/n, i = 0, 1, \dots, n - 1$ на $[a, b]$; xr, xi – рабочие массивы размерности n ; n – количество узлов сетки на $[a, b]$; eps – параметр, регулирующий точность вычисления интеграла при близких к нулю значениях $\omega_k x$; ip – параметр, определяющий аргумент экспоненты:

$ip = 1$ – вычисляем интеграл от $f(x) = e^{-i\omega x}$;

$ip = -1$ – вычисляем интеграл от $f(x) = e^{i\omega x}$.

В ы х о д н ы е п а р а м е т р ы : действительная и мнимая части значения интеграла в точках $\omega_k = 2\pi k / (b - a)$ находятся в массивах ar и ai соответственно.

(Процедура использует процедуру *procedure FF (var xr, xi, yr, yi: TMas; const n, ind: Integer);* для вычисления быстрого преобразования Фурье.)

Схема алгоритма процедуры IFF приведена на рисунке 44.

Текст процедуры:

```
procedure IFF (const a, b: Real; var ar, ai, xr, xi: TMas; const n: Integer; eps: Real; const ip: Integer);
var
  i, k : Integer;
  h, w, c, s, w1, w2, w3, w4, w5, a1, b1: Real;
begin
  h := (b - a)/n;
  {Вычисление быстрого преобразования Фурье процедурой FF}
  FF(ar, ai, xr, xi, n, ip);
  for i := 1 to n do
  begin
    k := i - 1;
    w := k*2*pi/(b - a);
    c := cos(w*a);
    s := sin(w*a);
    w := w*h*(b - a);
```

```

if w < eps then
begin
  w4 := h - 2*pi*pi*k*k*power(h,3)/power((b - a),2);
  w5 := 2*pi*k*h*h/(b - a);
  a1 := w4*ar[i] + ip*w5*ai[i];
  b1 := w4*ai[i] - ip*w5*ar[i];
end else
begin
  if k=0 then begin
    w1 :=0;
    w3 := 0
  end else begin
    w1 := sin(2*pi*k/n)*(b - a)/(2*pi*k);
    w2 := 2*power(sin(pi*k/n),2);
    w3 := w2*(b - a)/(2*pi*k);
  end;
  w2 := 2*power(sin(pi*k/n),2);
  a1 := w1*ar[i] + ip*w3*ai[i];
  b1 := w1*ai[i] - ip*w3*ar[i];
end;
ar[i] := a1*c + b1*s*ip;
ai[i] := b1*c - a1*s*ip;
if ip > 0 then
begin
  ar[i] := ar[i]*n*n;
  ai[i] := ai[i]*n*n;
end;
ar[i] := ar[i]/n;
ai[i] := ai[i]/n;
end;
end;

```

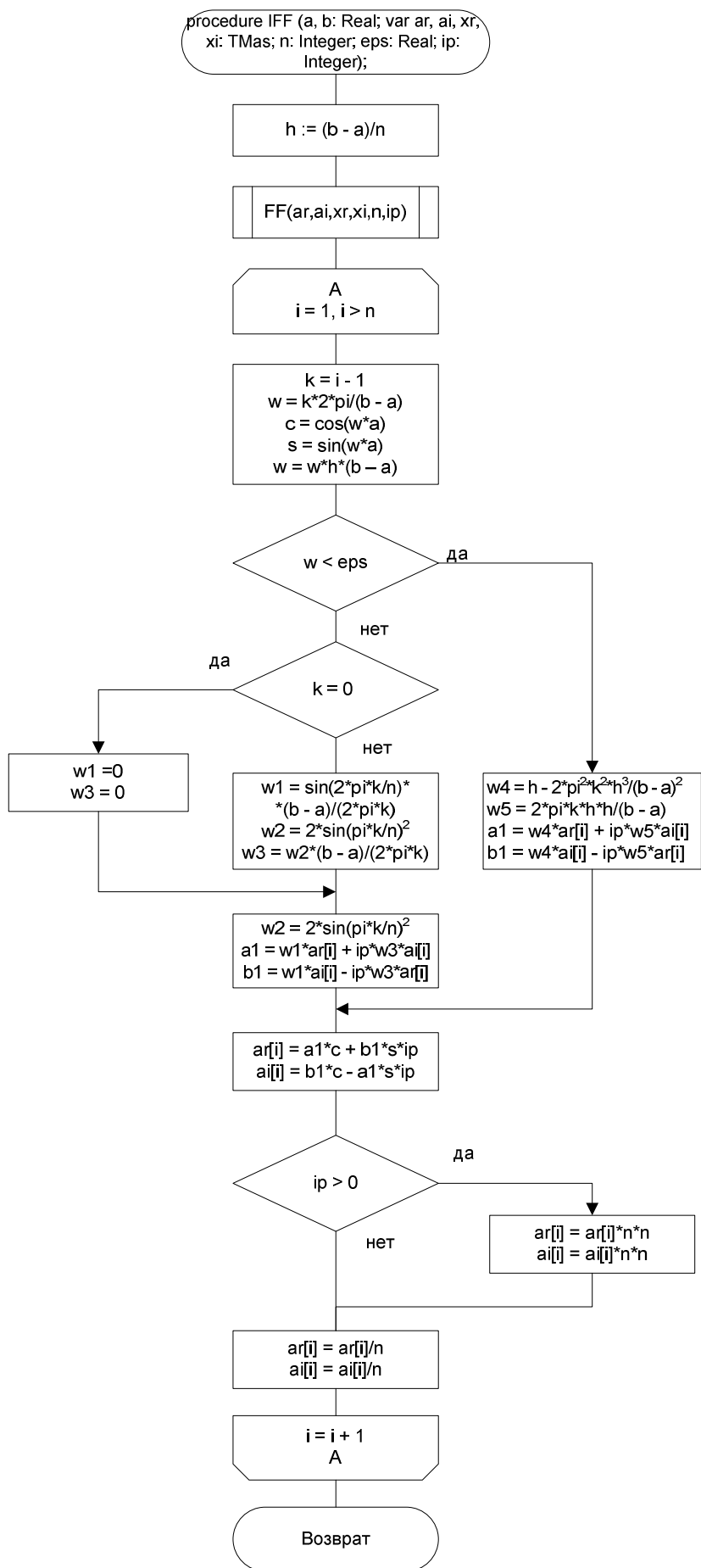


Рисунок 44 – Схема алгоритма процедуры IFF

Пример. Вычислить $\int_a^b e^{-x^2} e^{iax} dx$, используя квадратурную формулу с 512 узлами (при $\omega = \omega_k = \frac{2\pi k}{b-a}$, $k = 0, 1, \dots, n-1$, $n = 512$.)

Программа вычисления интеграла содержит вычисление значений $f(x)$ в узлах равномерной сетки из n узлов на $[a, b]$. Вычисленные значения размещаются в соответствующих массивах ar и ai . Далее вызывается процедура IFF.

Схема алгоритма приведена на рисунке 45.

```
unit Unit1;

interface

uses
  Windows, Messages, SysUtils, Variants, Classes, Graphics, Controls, Forms,
  Dialogs, StdCtrls, Grids, Math;

type
  TForm1 = class(TForm)
    sgOut: TStringGrid;
    btnRun: TButton;
    procedure btnRunClick(Sender: TObject);
  private
    { Private declarations }
  public
    { Public declarations }
  end;

var
  Form1: TForm1;

implementation

{$R *.dfm}

type TMas = array of Real;

procedure FF(var xr, xi, yr, yi: TMas; const n, ind: Integer);
var
  i, ni, j, jm, k, k1, m, mm, m2, log2, l, l1, it: Integer;
  xa, xb, w, si, co: Real;
begin
  k := n;
  log2 := 0;
  repeat
```

```

    k := k div 2;
    log2 := log2 + 1;
until (k < 2);
mm := 1;
{ cycl 1 }
for m := 1 to log2 do
begin
    m2 := mm*2;
    k1 := trunc(power(2,(log2 - m)) - 1);
    l1 := mm - 1;
    { cycl 2 }
    for k := 1 to k1 + 1 do
    begin
        { cycl 3 }
        for l := 1 to l1 + 1 do
        begin
            j := m2*(k - 1) + l;
            i := mm*(k - 1) + l;
            w := pi*(l - 1)/mm;
            si := ind*sin(w);
            co := cos(w);
            ni := trunc(power(2,(log2 - 1)) + i);
            jm := trunc(j + power(2, (m - 1)));
            xa := xr[ni]*co + xi[ni]*si;
            xb := xi[ni]*co - xr[ni]*si;
            yr[j] := xr[i] + xa;
            yi[j] := xi[i] + xb;
            yr[jm] := xr[i] - xa;
            yi[jm] := xi[i] - xb;
        end;
    { cycl 3 }
    end;
    { cycl 2 }
{ cycl 5 }
for it := 1 to n do
begin
    xr[it] := yr[it];
    xi[it] := yi[it];
end;
{ cycl 5 }
mm := m2;
end;
{ cycl 1 }
if ind < 0 then exit;
{ cycl 4 }
for i := 1 to n do
begin
    xr[i] := xr[i]/n;
    xi[i] := xi[i]/n;
end;
{ cycl 4 }
end;

```

```

procedure IFF (const a, b: Real; var ar, ai, xr, xi: TMas; const n: Integer; eps: Real; const ip: Integer);
var
  i, k : Integer;
  h, w, c, s, w1, w2, w3, w4, w5, a1, b1: Real;
begin
  h := (b - a)/n;
  {Вычисление быстрого преобразования Фурье программой FF}
  FF(ar, ai, xr, xi, n, ip);
  for i := 1 to n do
    begin
      k := i - 1;
      w := k*2*pi/(b - a);
      c := cos(w*a);
      s := sin(w*a);
      w := w*h*(b - a);
      if w < eps then
        begin
          w4 := h - 2*pi*pi*k*k*power(h,3)/power((b - a),2);
          w5 := 2*pi*k*h*h/(b - a);
          a1 := w4*ar[i] + ip*w5*ai[i];
          b1 := w4*ai[i] - ip*w5*ar[i];
        end else
        begin
          if k=0 then begin
            w1 :=0;
            w3 := 0
          end else begin
            w1 := sin(2*pi*k/n)*(b - a)/(2*pi*k);
            w2 := 2*power(sin(pi*k/n),2);
            w3 := w2*(b - a)/(2*pi*k);
          end;
          w2 := 2*power(sin(pi*k/n),2);
          a1 := w1*ar[i] + ip*w3*ai[i];
          b1 := w1*ai[i] - ip*w3*ar[i];
        end;
      ar[i] := a1*c + b1*s*ip;
      ai[i] := b1*c - a1*s*ip;
      if ip > 0 then
        begin
          ar[i] := ar[i]*n*n;
          ai[i] := ai[i]*n*n;
        end;
      ar[i] := ar[i]/n;
      ai[i] := ai[i]/n;
    end;
  end;

procedure TForm1.btnRunClick(Sender: TObject);
var
  n, i : Integer;

```

```

a, b, h, x, omg, eps : Real;
xr, xi, ar, ai : TMas;
begin
Form1.sgOut.Cells[0,0]:='№';
Form1.sgOut.Cells[1,0]:='омега';
Form1.sgOut.Cells[2,0]:='дейст-я часть';
Form1.sgOut.Cells[3,0]:='мнимая часть';
eps := 0.1E-6;
a := 0;
b := 20;
n := 512;
h := 20/n;
omg := 0;
SetLength(xr, n+1);
SetLength(xi, n+1);
SetLength(ar, n+1);
SetLength(ai, n+1);
for i := 1 to n do
begin
x := (i - 1)*h;
ar[i] := exp(-x*x);
ai[i] := 0;
end;
IFF(a,b,ar,ai,xr,xi,n,eps,1);
sgOut.RowCount := n + 1;
for i := 1 to n do
begin
omg := (i - 1)*2*pi/(b - a);
sgOut.Cells[0,i] := IntToStr(i);
sgOut.Cells[1,i] := FloatToStrF(omg,ffFixed,7,6);
sgOut.Cells[2,i] := FloatToStrF(ar[i],ffExponent,7,7);
sgOut.Cells[3,i] := FloatToStrF(ai[i],ffExponent,7,7);
end;
end;

end.

```

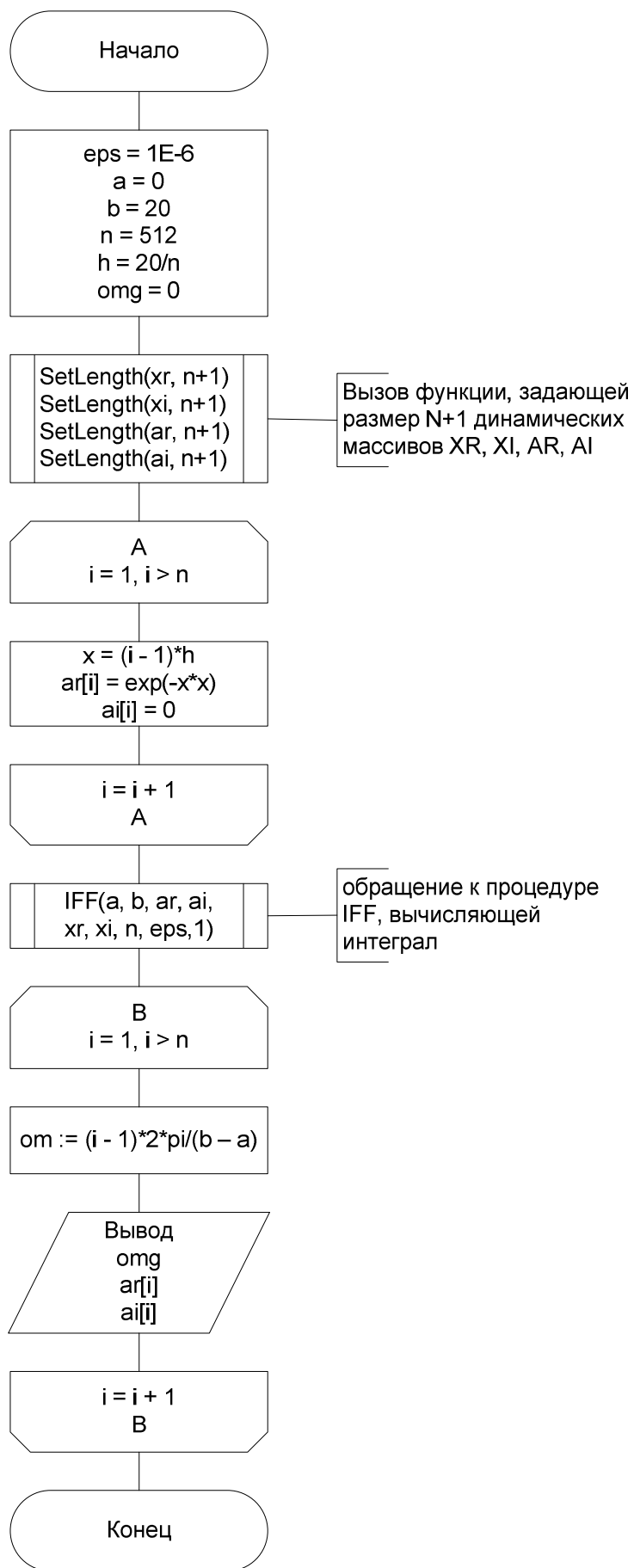


Рисунок 45 – Схема алгоритма вычисления интеграла.

Вычисления по программе привели к следующим результатам:

№	ω	Действительная часть	Мнимая часть
1	0,000000	9,057582E-1	0,000000E+0
2	0,314159	8,831889E-1	-1,599023E-1
3	0,628319	8,187761E-1	-3,042652E-1
4	0,942478	7,216180E-1	-4,205082E-1
5	1,256637	6,045151E-1	-5,011387E-1
6	1,570796	4,812212E-1	-5,445265E-1
7	1,884956	3,638563E-1	-5,542706E-1
8	2,199115	2,611314E-1	-5,375433E-1
9	2,513274	1,776797E-1	-5,030347E-1
10	2,827433	1,144008E-1	-4,591037E-1
11	3,141593	6,946244E-2	-4,125289E-1
12	3,455752	3,951928E-2	-3,679761E-1
13	3,769911	2,079204E-2	-3,280630E-1
14	4,084070	9,813039E-3	-2,937869E-1
15	4,398230	3,804643E-3	-2,650754E-1
16	4,712389	7,651842E-4	-2,412863E-1
17	5,026548	-6,231108E-4	-2,215746E-1
18	5,340708	-1,159402E-3	-2,051132E-1
...	...	...	...
493	154,566400	4,695544E-5	-6,865602E-3
494	154,880500	4,830774E-5	-6,908742E-3
495	155,194700	4,707561E-5	-6,966648E-3
496	155,508800	3,981786E-5	-7,044293E-3
497	155,823000	2,010035E-5	-7,147567E-3
498	156,137200	-2,309409E-5	-7,282281E-3
499	156,451300	-1,069578E-4	-7,451919E-3
500	156,765500	-2,556503E-4	-7,653768E-3
501	157,079600	-4,990090E-4	-7,873511E-3
502	157,393800	-8,676888E-4	-8,079315E-3
503	157,708000	-1,383714E-3	-8,217708E-3
504	158,022100	-2,046932E-3	-8,214580E-3
505	158,336300	-2,820312E-3	-7,984678E-3
506	158,650400	-3,619643E-3	-7,451095E-3
507	158,964600	-4,314501E-3	-6,572379E-3
508	159,278700	-4,745771E-3	-5,370084E-3
509	159,592900	-4,759961E-3	-3,945974E-3
510	159,907100	-4,253151E-3	-2,478437E-3
511	160,221200	-3,210886E-3	-1,193197E-3
512	160,535400	-1,728354E-3	-3,129204E-4

Варианты заданий

Вычислить $\int_{-\pi}^{\pi} f(x) e^{i\omega x} dx$

Таблица 10

Номер варианта	f(x)	Номер варианта	f(x)
1	$\sin x^2$	16	$\cos x^2$
2	$\sin(\sqrt{1+x^2})$	17	$\cos(\sqrt{1+x^2})$
3	$\operatorname{arctg}\left(\frac{1}{1+10x}\right)$	18	$\frac{1}{1+100x^2}$
4	$\arcsin\left(\frac{x+1}{10+x^2}\right)$	19	$\ln(x^2 - \pi^2 + 1)$
5	$\sin x \cos x^2$	20	$\cos\left(x + \frac{\pi}{2}\right) \sin x^2$
6	$\sin x \ln(x^2 + 1)$	21	$\sqrt{\pi^2 - x^2} \operatorname{arctg} x$
7	$\sqrt{\pi^2 - x^2} \sin x^2$	22	$(x^2 - \pi^2) \sin x^2$
8	$\sin x e^{-x^2}$	23	$e^{-10(x-\pi)^2}$
9	$\sqrt{\pi^2 - x^2} \operatorname{arctg}(x^2 + 7)$	24	$\sin x \operatorname{sh}(x^2 - \pi)$
10	$\operatorname{ch}(x^2 + 3) \sin x$	25	$\sin x \operatorname{ch}(\ln x + 2\pi)$
11	$\sqrt{x^2 + 18x + 20} \sin x^2$	26	$\frac{x^2 + \operatorname{ch} x}{1 + 65x^2}$
12	$\ln\left(\frac{x+12}{4} \operatorname{ch} x\right)$	27	$\sqrt{\pi^2 - x^2} \operatorname{ch} x$
13	$\cos(x - \pi)^2 \sin x$	28	$\sin(x - \pi)^2 \sin x$
14	$\ln(\sqrt{x^4 + 2} + \sin^2 x)$	29	$\frac{x^2 - 2x + 23}{10x + 100} \sin x$
15	$\sin x \cos x^2$	30	$\cos^2\left(x + \frac{\pi}{2}\right) \sin x^2$